
FRANKX · A2A PROTOCOL

The Agent-to-Agent (A2A) Protocol Specification

Machine-readable schemas, discovery manifests, and inter-agent communication

Frank Riemer · frankx.ai · 2026

The Agent Card

The Google [Agent-to-Agent \(A2A\) protocol](#) defines a JSON document called an **Agent Card** that describes an AI agent's identity, capabilities, and how to talk to it. It's served at `/.well-known/agent.json` on the agent's domain. Any A2A-compatible client can fetch it, understand the agent's skills, and know how to make a request.

This is the portability contract. Ship an Agent Card with your agent and:

- Other agents can discover and call yours.
- A multi-agent orchestrator can route requests to you.
- When you re-implement the agent on a different stack (Claude SDK, Google ADK, n8n), **the Agent Card stays the same.**

BCG's enterprise agent playbook treats the Agent Card as the discovery contract between agents in a mesh. It is, right now in 2026, the cleanest standardized spec for agent identity.

This guide walks every field of the A2A Agent Card, with examples, gotchas, and production notes.

Minimum valid Agent Card

```
{
  "name": "First Agent – Research Assistant",
  "description": "A research assistant that answers questions with cited sources.",
  "url": "https://first-agent-vercel-aisdk.vercel.app",
  "provider": {
    "name": "Frank Riemer",
    "url": "https://frankx.ai"
  },
  "version": "0.1.0",
  "capabilities": {
    "streaming": false,
    "pushNotifications": false,
    "stateTransitionHistory": false
  }
}
```

```

},
"authentication": {
  "schemes": []
},
"defaultInputModes": ["text"],
"defaultOutputModes": ["text", "application/json"],
"skills": [
  {
    "id": "research-with-sources",
    "name": "Research with sources",
    "description": "Given a question, searches the web and returns a structured Research of",
    "tags": ["research", "search"],
    "examples": ["What changed in the Vercel AI SDK v5 release?"],
    "inputModes": ["text"],
    "outputModes": ["application/json"]
  }
]
}

```

This is a valid A2A Agent Card. Every field is either required or strongly recommended. Let's walk them one by one.

Identity fields

name (required)

Human-readable name. One line, no branding fluff.

Good: "First Agent – Research Assistant", "Acme Support Triage", "Invoice OCR Agent" **Bad:** "AI-Powered Revolutionary Research Platform™", "agent-47"

description (required)

One paragraph, 100-400 characters. Describes what the agent does and — importantly — what it's for. Other agents and humans read this to decide whether to route work to you.

Good:

"A research assistant that answers factual questions with 2-4 cited sources. Returns structured output with confidence level and short caveats. Best for: tech/product research, comparison questions, up-to-date facts. Not for: medical, legal, or financial advice."

Bad:

"An AI agent that helps with things."

Be specific about **what it's for AND what it's not for**. Saving future callers from misrouted requests is an architectural courtesy.

`url` (required)

Base URL of the agent. This is where A2A clients send requests. Should be HTTPS in production.

Important: this is the **agent's URL**, not your company's homepage. If your agent lives at `https://agents.acme.com/support-triage`, that's the `url`.

`provider` (recommended)

Who's behind this agent. `{ name, url }`. Used for attribution, trust, and escalation paths.

```
"provider": {
  "name": "Acme Corp",
  "url": "https://acme.com"
}
```

`version` (recommended)

Semver string. Increment when you change behavior (new tools, new skills, breaking schema changes). Agents consuming your Card can pin to a known version.

`documentationUrl` (optional)

Link to human-readable docs. Saves callers from having to guess how your agent works.

Capability fields

`capabilities.streaming` (bool)

Does your agent support streamed responses? Most do in 2026; the default is `true` for modern agents. Set to `false` if your agent buffers the full response before replying.

`capabilities.pushNotifications` (bool)

Can the agent push status updates to a callback URL during long-running tasks? Typically `false` for simple agents. Set to `true` if you expose a `taskId` and allow clients to register webhook callbacks.

`capabilities.stateTransitionHistory` (bool)

Does the agent track its internal state history and expose it via an API? `false` for most first agents.

Auth fields

`authentication.schemes` (array)

How does the agent authenticate callers? Each entry is one of:

- `"none"` — public agent, no auth
- `"apiKey"` — header-based API key
- `"oauth2"` — OAuth 2.0
- `"bearer"` — bearer token

```
"authentication": {
  "schemes": ["apiKey"]
}
```

For a workshop-shipped agent, `"schemes": []` (public) is fine. For production, **add auth before you expose**. An open agent endpoint with a `web_search` tool is a DoS vector the first time someone notices it.

I/O fields

`defaultInputModes` (array)

Default MIME types or content modes the agent accepts. Typical values:

- `"text"` — plain text
- `"application/json"` — structured JSON input
- `"image/png"`, `"image/jpeg"` — multimodal
- `"audio/wav"` — for voice agents

defaultOutputModes (array)

What the agent returns. Same value set as inputs. For a structured-output agent, typically

`["text", "application/json"]`.

Skills — the most important field

A skill is a single capability the agent exposes. One agent, one or more skills. For a multi-capability agent, list each skill separately.

```
"skills": [  
  {  
    "id": "research-with-sources",  
    "name": "Research with sources",  
    "description": "Given a question, searches the web, synthesizes an answer, returns cited  
    "tags": ["research", "search", "summarization"],  
    "examples": [  
      "What changed in the Vercel AI SDK v5 release?",  
      "Compare Claude Sonnet 4.6 and GPT-5 for tool use."  
    ],  
    "inputModes": ["text"],  
    "outputModes": ["application/json"]  
  }  
]
```

Per-skill fields

- `id` (required) — stable identifier. Used in API calls to target a specific skill.
- `name` (required) — human-readable.

- `description` (required) — what this skill does and what it's for.
- `tags` (recommended) — an array of short strings for discovery. Think of these as categories an orchestrator uses to match skills to requests.
- `examples` (recommended) — 3-5 example inputs. Critical for other agents that want to decide if you're the right router target.
- `inputModes` / `outputModes` — override the agent-level defaults if this specific skill has different requirements.

Skill IDs are a contract

Once you publish a skill with `id: "research-with-sources"`, keep it stable. Other agents are routing on that ID. If you change it, you break their workflows. If you need to change behavior, **bump the version** of the agent and consider keeping the old skill ID as a deprecation alias for a release cycle.

Publishing the Agent Card

Two requirements:

1. **Serve it at** `/.well-known/agent.json` on the agent's domain. This is convention; don't put it anywhere else.
2. **Set the right headers.** Content-Type `application/json` and `Access-Control-Allow-Origin: *` (so cross-origin A2A clients can discover it).

In Next.js:

```
// next.config.mjs
export default {
  async headers() {
    return [
      {
        source: '/.well-known/agent.json',
        headers: [
          { key: 'Content-Type', value: 'application/json' },
          { key: 'Access-Control-Allow-Origin', value: '*' },
        ],
      },
    ],
  },
}
```

```
]
},
}
```

And the file itself goes in `public/.well-known/agent.json`.

Validation

Before shipping, validate your Card:

- **Online:** paste into a2a-protocol.org/validator (if available) or any JSON Schema validator against the A2A schema.
- **Local:** run the Agent Card through `zod` parsing:

```
import { z } from 'zod'

const AgentCardSchema = z.object({
  name: z.string(),
  description: z.string(),
  url: z.string().url(),
  provider: z.object({
    name: z.string(),
    url: z.string().url(),
  }).optional(),
  version: z.string(),
  capabilities: z.object({
    streaming: z.boolean(),
    pushNotifications: z.boolean(),
    stateTransitionHistory: z.boolean(),
  }),
  authentication: z.object({
    schemes: z.array(z.enum(['none', 'apiKey', 'oauth2', 'bearer'])),
  }),
  defaultInputModes: z.array(z.string()),
  defaultOutputModes: z.array(z.string()),
  skills: z.array(z.object({
```



```
id: z.string(),
name: z.string(),
description: z.string(),
tags: z.array(z.string()).optional(),
examples: z.array(z.string()).optional(),
inputModes: z.array(z.string()).optional(),
outputModes: z.array(z.string()).optional(),
}))}.min(1),
})

// parse your card
AgentCardSchema.parse(agentCardJson)
```

If it parses, you're good.

The Oracle Open Agent Specification connection

The Agent Card is one half of the portability story. The other half is the [Oracle Open Agent Specification \(OAS\)](#) — a framework-agnostic YAML/JSON spec that describes **the agent's internals** (tools, memory, workflows), not just its external interface.

Relationship:

- **Agent Card** = external contract. What callers see. Fields: name, description, skills, capabilities.
- **OAS** = internal definition. What developers maintain. Fields: tools, memory stores, workflows, model routing, observability.

In practice: your Agent Card is a **subset** of what an OAS file contains. Generate the Agent Card from the OAS file and both stay in sync.

This is especially valuable for enterprise deployments where one team owns the OAS definition and another team consumes the Agent Card. It's the same pattern as OpenAPI specs generating client libraries across languages.

The [enterprise branch of the workshop \(B6\)](#) covers OAS in depth. For your first agent, the Card alone is enough.

Gotchas

1. Don't put secrets in the Card

The Card is public. API keys, internal URLs, customer IDs do not belong in it. The `authentication` field describes the auth **scheme** — it never contains the credentials themselves.

2. `url` must be reachable

If you put `https://internal.corp.com/my-agent` in the Card and publish it on the public internet, any A2A client that tries to call your agent will fail. Keep the Card and the agent's reachability aligned.

3. Skill IDs aren't display strings

`"id": "research-with-sources"` is a machine identifier — kebab-case, stable, meaningful. `"name": "Research with sources"` is what humans see. Don't conflate them.

4. `examples` are part of the contract

Callers use `examples` to decide if your skill is the right match for their intent. If you ship misleading examples, you get misrouted requests. Pick examples that are representative of the actual skill.

5. CORS is a common gotcha

If another agent on a different origin tries to fetch your Agent Card and gets a CORS error, they can't discover you. Set `Access-Control-Allow-Origin: *` on `/.well-known/agent.json` unless you have a specific reason not to.

Production checklist

Before exposing an Agent Card in production:

- ☐ All required fields populated (`name` , `description` , `url` , `version` , `capabilities` , `authentication` , `defaultInputModes` , `defaultOutputModes` , `skills`)
- ☐ At least one skill with `id` , `name` , `description` , and 2+ examples
- ☐ `url` is reachable from the public internet
- ☐ Served at `/.well-known/agent.json` with `Content-Type: application/json` and CORS headers

- ☐ Validated against the A2A schema
 - ☐ No secrets in the Card
 - ☐ Auth scheme is documented — "none" is acceptable for open agents but means you **must** add rate limits at a network layer
 - ☐ Version bumped if behavior changed since last publish
 - ☐ Documentation URL points to real docs, not your company homepage
-

What to read next

- [The six primitives of an AI agent](#) — the mental model
- [First Agent Primer](#) — ship your first agent end-to-end
- [Google A2A protocol](#) — the authoritative spec
- [Build Your First AI Agent workshop](#) — live version

When you publish your Card, drop the URL at `frankx.ai/workshops/build-first-ai-agent` (gallery section — coming soon) and we'll feature agents with well-crafted Cards.